Sistema de exámenes médicos Cuidarte+

Informe de seguridad y calidad en el desarrollo de software

ISY1102 - Seguridad y calidad en el desarrollo de software

Sección 003D

Carrera: Ingeniería en Informática

Docente: Luis Yagi

Integrantes

Benjamín Ibáñez
Gerardo Vera González

Evaluación 2 — Encargo (EP3)

Fecha: 20-10-2026 — Versión 1.0

ÍNDICE

1. Introducción 3
1.1 Ajustes incorporados desde la Evaluación 1 3
2. Criterios de calidad 5
2.1 Calidad del producto (ISO/IEC 25010) 5
2.2 Seguridad 5
2.3 Cumplimiento normativo 6
3. Plan de pruebas 8
3.0 Objetivo y alcance 8
3.1 Estrategia de pruebas 8
3.2 Tipos de prueba 9
3.3 Criterios de aceptación 11
3.4 Herramientas y recursos utilizados 11
4. Diseño de casos de prueba 14
4.1 Trazabilidad y cobertura de requerimientos 14
4.2 Casos de prueba documentados 17
5. Ejecución de pruebas 22
5.1 Resumen de ejecución 22
5.2 Detalle de ejecución de casos 22
5.3 Criticidad de incidentes 23
5.4 Análisis estático con SonarQube: incidencias priorizadas 27
6. Análisis ético y normativo de los resultados 28
6.1 Legal y normativo (contexto chileno) 28
6.2 Ética 29
7. Conclusiones, recomendaciones y acciones pendientes 30
Anexo A. Evidencias de ejecución 31
A.1 Casos CP001–CP004 (capturas de interfaz y API) 31
A.2 Casos CP005–CP008 (respuestas de API) 34
Anexo B. Fichas de las 25 incidencias priorizadas de SonarQube 35

1. Introducción

Cuidarte+ es una plataforma web multicapa desarrollada por CleaLAB Spa para centros de salud, orientada a la gestión de pacientes, exámenes médicos y documentos clínicos. Su arquitectura integra un frontend en React (Material UI y Tailwind), un backend API RESTful en Node.js/Express y una base de datos PostgreSQL. El sistema gestiona datos sensibles de salud — fichas clínicas, diagnósticos y documentos adjuntos — por lo que la calidad del software no puede evaluarse solo por su funcionamiento: la seguridad y el cumplimiento normativo son pilares fundamentales de la calidad en este contexto.

Este informe corresponde a la segunda iteración del caso semestral (Evaluación Parcial 3) y consolida el trabajo de aseguramiento de la calidad realizado por el equipo durante la Experiencia 2: el Plan Maestro de pruebas, el diseño y ejecución de casos de prueba, la revisión manual del código fuente guiada por OWASP Top 10 y el análisis estático automatizado con SonarQube. El objetivo es determinar, con evidencia trazable, si el sistema puede liberarse a producción y bajo qué condiciones, identificando las vulnerabilidades y problemas de calidad encontrados con su criticidad correspondiente.

El informe se estructura de la siguiente forma: la sección 2 define los criterios de calidad, seguridad y cumplimiento normativo que el software debe satisfacer; la sección 3 presenta el plan de pruebas (estrategia, tipos de prueba, criterios de aceptación y herramientas); la sección 4 documenta el diseño de los casos de prueba; la sección 5 registra la ejecución y los incidentes con su criticidad; la sección 6 desarrolla el análisis ético y normativo; y la sección 7 cierra con conclusiones, recomendaciones y el veredicto de certificación.

1.1 Ajustes incorporados desde la Evaluación 1

La Evaluación Parcial 1 obtuvo 88/100 puntos (nota 6,1). A partir de los puntajes por indicador, el equipo identificó tres brechas y las incorporó en este informe:

Indicador EP1	Puntaje	Ajuste incorporado en este informe
Identifica la importancia de la calidad, la seguridad y el cumplimiento legal (15%)	12/15 — omisiones y ejemplos limitados	Se integraron ejemplos concretos del sistema: credenciales en texto plano en auth.js, revocación de sesión ausente confirmada en ejecución (CP006), RUT aceptado sin dígito verificador, y resultados reales de SonarQube.

Indicador EP1	Puntaje	Ajuste incorporado en este informe
Reconoce los elementos que componen un plan de pruebas y su vinculación con requerimientos y normativas (25%)	20/25 — apartados incompletos y vínculos poco detallados	El plan ahora incluye todos los apartados (objetivo, alcance, estrategia, tipos, criterios de aceptación, herramientas, recursos humanos/técnicos, entorno y cronograma), con cada tipo de prueba vinculado explícitamente a su requisito (RQ/RF/NFR), regla de negocio y riesgo priorizado (RS-01 a RS-04).
Analiza la cobertura, pertinencia y coherencia del plan frente a estándares técnicos y legales (20%)	16/20 — análisis superficial	Se incorporó la matriz de trazabilidad requisito → caso → estado de cobertura de la sección 4.1, y un análisis crítico de coherencia del plan contra ISO/IEC 25010, OWASP Top 10 y las leyes chilenas 19.628 y 20.584, distinguiendo lo cumplido, lo parcial y lo no ejecutado con su justificación.

2. Criterios de calidad

Los criterios que el software debe cumplir se derivan del documento de especificación de requerimientos (ERS Caso B), del contexto de negocio (datos de salud) y de los estándares de referencia. Se organizan en tres grupos: calidad general del producto, seguridad y cumplimiento normativo.

2.1 Calidad del producto (ISO/IEC 25010)

- Adecuación funcional: el sistema debe implementar correctamente los requerimientos RF-1 a RF-5 del ERS (autenticación y sesión, registro de usuarios, gestión de exámenes, visualización de información médica y auditoría), con validaciones que impidan guardar datos inválidos. Ejemplo concreto: el registro de pacientes debe rechazar con HTTP 400 un email sin @ o un nombre de 1 letra, sin alterar la base de datos (verificado en CP001–CP004).
- Eficiencia de desempeño: operaciones CRUD simples bajo 300 ms (NFR-PERF-1) y carga de documentos bajo 10 segundos (RF-4). Ejemplo concreto: se detectó un retardo intencional de 5 segundos en los endpoints de listado (middleware delay.js), confirmado en ejecución con 5.036 ms en CP007 y 5.050 ms en la carga de CP008, lo que impide cumplir la meta.
- Confiabilidad y estabilidad: continuidad operacional del sistema (NFR-DIS-1, disponibilidad 99,5% mensual) y ausencia de fallas bloqueantes en los flujos críticos. Ejemplo: los flujos probados (registro, control de acceso, documentos) no presentaron fallas funcionales bloqueantes.
- Usabilidad y accesibilidad: mensajes de error claros que indiquen qué campo falló y cómo corregirlo (NFR-USAB-3), flujos claros para registro y visualización de exámenes (NFR-USAB-2) y compatibilidad con navegadores modernos y dispositivos móviles (NFR-COMPAT-1/2). Ejemplo: el formulario de pacientes muestra mensajes por campo (evidencias CP002 y CP004).
- Mantenibilidad: código interno mantenible, medido con análisis estático. Ejemplo concreto: SonarQube 9.9.8 reportó 8 issues críticos de complejidad cognitiva (S3776), 14,5% de duplicación y una deuda técnica de 465 minutos sobre 10.133 líneas de código, lo que encarece futuras modificaciones.

2.2 Seguridad

- Protección de datos: las contraseñas deben almacenarse con hash seguro (NFR-SEG-2) y los datos sensibles protegidos en reposo y tránsito (NFR-SEG-1 y NFR-SEG-3). Ejemplo concreto: el código compara y almacena contraseñas en texto plano (auth.js), hallazgo calificado como crítico (H-01).
- Control de acceso: autorización por rol en el backend (NFR-SEG-4) según las reglas de negocio RB-1 a RB-3: el paciente solo lee, el médico gestiona fichas y el administrador gestiona todo. Ejemplo concreto: una paciente que intentó editar un examen recibió 403 Forbidden sin cambios en la base de datos (CP007).
- Gestión de sesión: manejo mediante token JWT con revocación al cerrar sesión (NFR-SEG-9, RF-1.2). Ejemplo concreto: el token reutilizado tras cerrar sesión siguió siendo aceptado con 200 (CP006, hallazgo H-02), incumpliendo el criterio del ERS.
- Integridad de la información: los documentos descargados deben ser íntegros respecto del original y las identificaciones de pacientes confiables. Ejemplos concretos: la descarga de CP008 verificó integridad con SHA256 idéntico entre el archivo subido y el descargado; en cambio, el RUT se valida solo por formato sin dígito verificador (hallazgo H-03), permitiendo fichas asociadas a identidades inconsistentes.
- Manejo de errores seguro: los errores internos no deben exponer información técnica al cliente (NFR-SEG-6, gestión de registros; OWASP A05/A09). Ejemplo: mensajes con err.message se devuelven al cliente en varias rutas (hallazgo H-07).

2.3 Cumplimiento normativo

- Ley 19.628 (protección de la vida privada): los datos de salud son datos sensibles; el sistema debe resguardar su confidencialidad. Los hallazgos H-01, H-02 y H-04 afectan directamente este deber.
- Ley 20.584 (derechos y deberes del paciente): la ficha clínica es reservada, su acceso debe estar controlado y trazado. La auditoría existente es una fortaleza, pero la falta de revocación de sesión (H-02) debilita el control de acceso y la identificación errónea de pacientes (H-03) compromete la correcta asociación de la ficha.
- NFR-SEG-8 del ERS: cumplimiento de normativas de protección de datos personales, verificado mediante revisión manual OWASP Top 10 y análisis estático.
- OWASP Top 10 (2021): el sistema debe evitar los riesgos categorizados; los hallazgos se clasificaron según A01 (control de acceso roto — H-04), A02 (fallos criptográficos —

H-01, H-02), A05 (configuración incorrecta — H-06) y A07 (fallos de identificación y autenticación — H-02, H-03).

3. Plan de pruebas

3.0 Objetivo y alcance

Objetivo: verificar que Cuidarte+ garantice la confidencialidad, integridad, disponibilidad y rendimiento en la gestión de datos médicos y usuarios, asegurando el cumplimiento de controles de seguridad OWASP y de control de acceso por rol, con trazabilidad de evidencia para cada caso ejecutado.

Incluye: módulos de autenticación (JWT), control de acceso (RBAC), registro y validación de pacientes (POST /pacientes), gestión de exámenes médicos, carga y descarga de documentos, registro de auditoría y revisión de seguridad API/Web (OWASP Top 10 manual) más análisis estático automatizado.

No incluye en esta iteración: integración con sistemas externos de salud, pasarelas de pago, despliegue en producción, pruebas de carga con 200 usuarios concurrentes, DAST exhaustivo y la corrección del código fuente (los defectos se registran y recomiendan, no se corrigen, según la pauta de la actividad). La matriz de la sección 4.1 declara requisito por requisito qué quedó fuera de la verificación y por qué.

3.1 Estrategia de pruebas

Enfoque mixto (manual y automatizado). Las pruebas manuales cubren los flujos críticos de interfaz y API (interfaz web + Swagger/Postman), la revisión estática del código guiada por OWASP Top 10 y la verificación de mensajes de usabilidad. Las herramientas automatizadas cubren el análisis estático del código (SonarQube) y la reproducibilidad del entorno (Docker).

Etapas del proceso: (1) reconocimiento del sistema levantado en Docker; (2) diseño de casos con técnicas formales (partición de equivalencia y valores límite) sobre las reglas de validación reales del código; (3) ejecución funcional y de seguridad con registro de evidencia; (4) análisis estático automatizado del código fuente; (5) consolidación de hallazgos, clasificación de criticidad y emisión del veredicto.

Priorización: basada en los riesgos priorizados del Plan Maestro — RS-01 escalación de privilegios vía token JWT (alta), RS-02 fuga de datos por falta de sanitización (alta), RS-03 lentitud ante concurrencia (media), RS-04 ausencia de auditoría en operaciones críticas (media). La seguridad de sesión y datos se prueba primero; luego las validaciones funcionales y el rendimiento.

Criterios de entrada: contenedores Docker levantados (frontend 3333, backend 4444, PostgreSQL 15442), usuarios de prueba con roles médico y paciente disponibles, colección Swagger/Postman operativa y datos sintéticos cargados. Criterios de salida: 100% de los casos diseñados ejecutados con evidencia archivada, hallazgos clasificados por criticidad y veredicto de certificación emitido.

3.2 Tipos de prueba

Se aplicaron los siguientes tipos de prueba, cada uno vinculado al requisito y riesgo que justifica su inclusión. La columna «Aplicación» distingue lo ejecutado de lo no ejecutado: un tipo marcado como «no aplicada» no se considera aprobado, y su motivo queda registrado en la propia tabla:

Tipo de prueba	Qué verifica	Vinculación	Herramienta o técnica	Aplicación
Funcionales y de borde	Registro y listado de pacientes con datos válidos, inválidos, duplicados y de borde; carga y descarga de documentos con integridad.	RQ-07 (RF-2), RQ-03 (RF-3/RF-4); riesgo RS-02	Interfaz web + Swagger/Postman; partición de equivalencia y valores límite	Aplicada
Seguridad — sesión y acceso	Login con JWT, control por rol y propiedad (401/403), revocación de sesión al cerrar sesión.	RQ-01 (RF-1, NFR-SEG-9), RQ-02 (RB-1 a RB-3, NFR-SEG-4); riesgos RS-01, RS-02	API (curl/Postman) + revisión de código	Aplicada
Seguridad — análisis estático	Almacenamiento de contraseñas, secreto JWT, configuración de archivos y CORS; calidad interna del código (bugs, code smells, vulnerabilidades).	NFR-SEG-1 a NFR-SEG-9; OWASP Top 10	Revisión manual OWASP Top 10 + SonarQube 9.9.8	Aplicada
Rendimiento	Tiempos de respuesta reales de CRUD y de carga/descarga de documentos (incluye retardo intencional de 5 s detectado).	RQ-04 (NFR-PERF-1 a 3); riesgo RS-03	Medición con API (curl) + DevTools; JMeter planificado	Aplicada

Tipo de prueba	Qué verifica	Vinculación	Herramienta o técnica	Aplicación
Usabilidad	Claridad de los mensajes de error del formulario de pacientes (qué falló y cómo corregirlo).	RQ-07 (NFR-USAB-3)	Lista de chequeo de interfaz	Aplicada
Integración / API	Comunicación frontend–backend–base de datos: contenedores levantados, /docs responde y los endpoints del frontend existen.	Base operativa de toda la ejecución	docker ps, docker compose logs, Swagger (/docs, /openapi.json)	Aplicada
Regresión — no aplicada	Confirmar que un cambio no rompe lo ya verificado (CP001–CP008 volverían a correr tras cada corrección).	NFR-DIS-2 (actualizaciones con ventana de mantenimiento y migración segura): un cambio en el código exige re-verificar lo ya aprobado.	No aplica hoy: no hubo corrección de código entre jornadas (el backend es vulnerable a propósito y no se tocó). Correría tras remediar H-01 a H-04.	No aplicada
Aceptación (UAT) — no aplicada	Validar el sistema con usuarios reales del proceso (médicos, administrativos, pacientes) contra sus tareas reales.	NFR-USAB-1 y NFR-USAB-2; criterios de negocio del ERS.	No aplicada: requiere usuarios reales del centro de salud; el equipo no representa a esos perfiles. Queda como acción pendiente.	No aplicada
Compatibilidad — no aplicada	Comportamiento entre navegadores y dispositivos (Chrome/Firefox/Edge, escritorio/tablet/móvil).	NFR-COMPAT-1 y NFR-COMPAT-2 (navegadores y diseño responsivo).	No aplicada: se ejecutó solo en Chrome de escritorio. Sin matriz de navegadores ni dispositivos reales.	No aplicada
Accesibilidad — no aplicada	Uso del sistema con tecnologías de asistencia (lector de pantalla, navegación por teclado, contraste).	NFR-USAB-4; criterio de inclusión para un sistema de salud.	No aplicada: no hay herramientas de auditoría (axe, Lighthouse) configuradas en el entorno.	No aplicada
Carga y estrés — no aplicada	Comportamiento del sistema bajo demanda concurrente y volumen de datos.	NFR-PERF-1 a NFR-PERF-3; riesgo RS-03.	No aplicada: no existe entorno de carga ni datos de volumen; solo mediciones puntuales (CP007: 5.036 ms con delay; CP008: descarga 37 ms).	No aplicada

Justificación de la selección: las pruebas funcionales y de borde son el flujo crítico del negocio y concentran las reglas de validación del módulo propio del equipo (registro de pacientes). Las pruebas de seguridad responden a que el sistema maneja datos de salud y su código es vulnerable a propósito (contraseñas en texto plano, sesiones sin revocación), por lo que su

verificación es prioritaria (RS-01/RS-02). El análisis estático automatizado complementa la revisión manual al auditar el código completo de forma sistemática (IL 2.3). Las mediciones de rendimiento evidencian el retardo artificial antes de comprometer metas formales. La prueba de integración es la base: si un contenedor cae, nada funciona. La usabilidad se incluye porque el sistema es usado por pacientes y personal de salud con poco tiempo.

3.3 Criterios de aceptación

Criterios transversales para considerar aprobado un caso de prueba, derivados del ERS y de las reglas de negocio:

- Seguridad: ninguna falla de severidad crítica o alta sin registrar; toda solicitud a recursos protegidos sin token responde 401 y con rol o propiedad incorrectos responde 403; el login entrega un token JWT válido con expiración de 2 horas; cerrar sesión debe invalidar el token (RF-1.2).
- Funcionalidad y negocio: ante datos inválidos, el sistema responde 400 con detalle del campo y no guarda nada; ante duplicados (RUT o email) responde 400 sin agregar filas; un registro válido responde 201, aparece en el listado y genera registro de auditoría (RF-5.1).
- Rendimiento: la carga y descarga de documentos no debe exceder 10 segundos (RF-4); las metas de <300 ms en el 95% de los CRUD y 200 usuarios concurrentes quedan reevaluadas para la próxima iteración tras retirar el retardo de 5 s (H-05).
- Integridad: el documento descargado debe ser idéntico al subido (verificación SHA256).
- Criterios del ERS: login con token y logout que invalida la sesión (RF-1.2) — no se cumple (CP006); subida y descarga de documentos en menos de 10 segundos (RF-4) — se cumple (CP008: 5,087 s en total); CRUD de exámenes con validación y eliminación lógica confirmada (RF-3) — no verificado; y auditoría de seguridad con checklist y pruebas de penetración sin hallazgos críticos (NFR-SEG) — no se cumple: no se ejecutaron pruebas de penetración y H-01 es crítico.
- Análisis estático: Quality Gate de SonarQube en estado OK (sin condiciones configuradas; ver 5.1) y cero vulnerabilidades nuevas; todo hotspot o issue crítico se registra como hallazgo con su recomendación.

3.4 Herramientas y recursos utilizados

Herramientas tecnológicas (con su fundamento):

- Docker Compose: levanta el sistema completo (frontend, backend y PostgreSQL) en un entorno reproducible y aislado, condición para que toda evidencia sea repetible.
- Swagger (/docs) y Postman/curl: diseño y ejecución de pruebas funcionales y de seguridad sobre la API RESTful, con documentación viva de los contratos.
- SonarQube Community 9.9.8: análisis estático del código fuente (SAST) para auditar calidad interna y vulnerabilidades conforme a criterios técnicos y normativos (IL 2.3); seleccionado por ser estándar de la industria, gratuito y con perfiles OWASP.
- Chrome DevTools: inspección de red para verificar tiempos de respuesta y comportamiento del frontend.
- SHA256 (certutil): verificación de integridad de documentos entre subida y descarga (CP008).

Recursos humanos: el equipo asumió los roles de Líder QA (coordinación y diseño de casos), Ejecutor (ejecución y registro), Responsable de evidencia (captura y archivo) y Relator (bitácora e informe), distribuidos entre Benjamín Ibáñez y Gerardo Vera.

Recursos técnicos y entorno: tres contenedores Docker en equipo local — frontend (http://localhost:3333), backend/API (http://localhost:4444, Swagger en /docs) y PostgreSQL 15 (puerto 15442) — con datos sintéticos; navegador Chrome; código fuente del caso en el repositorio del equipo.

Plan de avance (cronograma ejecutado):

Actividad	**Responsable**	**Fecha**	**Evidencia**
Levantar y reconocer el sistema	Benjamín Ibáñez	22-09-2026	Contenedores Docker levantados
Definir alcance y requisitos	Ambos	23-09-2026	Secciones de objetivo y requisitos del Plan Maestro
Diseñar estrategia y riesgos	Gerardo Vera	24-09-2026	Matriz de riesgos y tabla de estrategia
Diseñar casos iniciales (CP001–CP008)	Ambos	29-09-2026	Guía 2.2 y Sección 7 del Plan Maestro
Ejecutar CP001–CP004 con evidencia	Benjamín Ibáñez	06-10-2026	Bitácora 2.3 y capturas Pass
Ejecutar CP005–CP008 con evidencia	Benjamín Ibáñez	16-10-2026	Resumen de ejecución y JSON de respuestas
Análisis estático SonarQube	Benjamín Ibáñez	16-10-2026	Log del escáner, métricas y issues

Actividad	Responsable	Fecha	Evidencia
Revisión y consolidación de este informe	Ambos	20-10-2026	Informe EP3

4. Diseño de casos de prueba

Los casos fueron diseñados con la técnica de partición de equivalencia y valores límite, seleccionada porque el módulo de registro de pacientes es un formulario con rangos y formatos (nombre y apellido de 2 a 50 letras, RUT con formato, email válido de hasta 255 caracteres, fecha no futura, sexo M/F/O), y se complementó con casos de seguridad sobre sesión y control de acceso. Fuente de las reglas: BACKEND/src/validations/pacientes.js. A continuación se documentan los 8 casos representativos de los distintos tipos de requerimientos del ERS:

4.1 Trazabilidad y cobertura de requerimientos

El ERS Caso B contiene 37 requisitos verificables: 17 funcionales (RF-1.1, RF-1.2, RF-2.1 a RF-2.7, RF-3.1 a RF-3.3, RF-4.1 a RF-4.4 y RF-5.1) y 20 no funcionales (NFR-SEG-1 a 9, NFR-PERF-1 a 3, NFR-USAB-1 a 4, NFR-DIS-1 y 2, NFR-COMPAT-1 y 2). La tabla siguiente los inventaría todos y declara, para cada uno, cómo se verificó, con qué caso o artefacto y en qué estado quedó. Los estados son: Verificado (caso ejecutado con evidencia y resultado Pass), No conforme (la verificación arrojó un hallazgo con severidad), Parcial (verificación indirecta, incompleta o con salvedad) y No verificado (fuera del alcance declarado en la sección 3.0).

Requisito ERS	Qué exige	Verificación, caso o artefacto	Estado
RF-1.1	Autenticación con nombre_usuario y contraseña	CP005: login 200 y uso del token en GET /pacientes	**Verificado**
RF-1.2	Cierre de sesión y revocación de tokens	CP006: el token previo sigue aceptado (200; esperado 401) — H-02	**No conforme**
RF-2.1	Registro de usuario con rol paciente por defecto	Revisión de auth.js (/registro); sin caso ejecutado	**Parcial**
RF-2.2	Médico o administrador registra pacientes	CP001–CP004: 201 y 400 por email, duplicado y borde (la identidad no se valida: H-03)	**Verificado**
RF-2.3	Médico o administrador edita un paciente	No ejecutado en esta iteración	**No verificado**
RF-2.4	Médico o administrador visualiza un paciente	CP001: el paciente creado aparece en el listado	**Parcial**
RF-2.5	Administrador visualiza los usuarios registrados	No ejecutado en esta iteración	**No verificado**
RF-2.6	Administrador edita usuarios y asigna roles	No ejecutado en esta iteración	**No verificado**
RF-2.7	Administrador elimina usuarios	No ejecutado en esta iteración	**No verificado**

Requisito ERS	Qué exige	Verificación, caso o artefacto	Estado
RF-3.1	Médico registra exámenes de un paciente	CP007: examen creado (201) como precondición	**Parcial**
RF-3.2	Médico edita el examen y sus documentos	CP007 solo prueba el 403 del paciente; la edición médica no se ejecutó	**Parcial**
RF-3.3	Médico elimina examen con confirmación y borrado lógico	No ejecutado en esta iteración	**No verificado**
RF-4.1	Paciente visualiza sus exámenes	No ejecutado (CP008 usa sesión de médico)	**No verificado**
RF-4.2	Paciente descarga los documentos de sus exámenes	CP008 descarga con sesión de médico; no con paciente	**Parcial**
RF-4.3	Médico adjunta documentos PDF/DOC/PPT/JPG/PNG	CP008: subida 201 de un PDF	**Verificado**
RF-4.4	Documentos en storage interno con referencia en BD	Código: bytes en la columna documento (memStorage), sin storage interno	**Parcial**
RF-5.1	Auditoría de acciones críticas (alta, edición, borrado, subida, descarga)	Inserts de auditoría en auth, pacientes, exámenes y documentos; CP001 sin captura del registro	**Parcial**
NFR-SEG-1	TLS obligatorio en todas las comunicaciones	Entorno local por HTTP; no verificado	**No verificado**
NFR-SEG-2	Contraseñas con hashing seguro (bcrypt)	H-01: comparación y almacenamiento en texto plano	**No conforme**
NFR-SEG-3	Cifrado de datos sensibles en reposo	No verificado	**No verificado**
NFR-SEG-4	Control de acceso por rol en el backend	CP007: 403 al rol paciente; 401 sin token	**Verificado**
NFR-SEG-5	Protección CSRF, XSS, SQLi y saneamiento de entrada	Revisión de código: consultas parametrizadas; sin DAST	**Parcial**
NFR-SEG-6	Gestión de log segura	H-07: err.message expuesto al cliente; logs en consola	**Parcial**
NFR-SEG-7	Políticas de respaldo y recuperación	No verificado	**No verificado**
NFR-SEG-8	Cumplimiento de normativas de datos personales	Revisión OWASP y análisis legal de la sección 6	**Parcial**
NFR-SEG-9	Manejo de sesión con token JWT	CP005 Pass y CP006 Fail (H-02)	**Parcial**
NFR-PERF-1	CRUD simples bajo 300 ms	H-05: 5.036 ms en CP007 por el retardo de 5 s	**No conforme**
NFR-PERF-2	Página principal bajo 2 s en conexión móvil	No verificado	**No verificado**
NFR-PERF-3	200 usuarios concurrentes	Fuera de alcance (declarado en 3.0)	**No verificado**

Requisito ERS	Qué exige	Verificación, caso o artefacto	Estado
NFR-USAB-1	Tipografía legible, contraste y controles grandes	Sin pruebas con usuarios (adultos mayores)	**No verificado**
NFR-USAB-2	Flujos claros de registro y visualización	CP002 y CP004: mensajes por campo en el formulario	**Parcial**
NFR-USAB-3	Mensajes de error amigables y ayuda contextual	CP002 y CP004: 400 con detalle del campo	**Verificado**
NFR-USAB-4	Acceso escalado: primero el paciente, luego sus exámenes	No verificado	**No verificado**
NFR-DIS-1	Disponibilidad de 99,5% mensual	Sin monitoreo ni serie histórica	**No verificado**
NFR-DIS-2	Actualizaciones con ventana y migración segura	No verificado	**No verificado**
NFR-COMPAT-1	Últimas 2 versiones de Chrome, Firefox, Safari y Edge	Solo Chrome en el entorno de pruebas	**No verificado**
NFR-COMPAT-2	Diseño responsivo en móviles y tablets	No verificado	**No verificado**

Resultado de cobertura: 5 requisitos verificados con caso y evidencia (RF-1.1, RF-2.2, RF-4.3, NFR-SEG-4 y NFR-USAB-3), 3 no conformes (RF-1.2 y NFR-SEG-2 por H-01/H-02; NFR-PERF-1 por H-05), 12 parciales y 17 no verificados. El inventario queda cubierto al 100% en términos de trazabilidad —ningún requisito sin decisión documentada—, mientras que la verificación efectiva con evidencia alcanza 8 de 37 requisitos (21,6%) y la verificación indirecta otros 12 (32,4%). Los 17 no verificados son RF-2.3, RF-2.5, RF-2.6, RF-2.7, RF-3.3 y RF-4.1; NFR-SEG-1, NFR-SEG-3 y NFR-SEG-7; NFR-PERF-2 y NFR-PERF-3; NFR-USAB-1 y NFR-USAB-4; NFR-DIS-1 y NFR-DIS-2; y NFR-COMPAT-1 y NFR-COMPAT-2. Corresponden a funciones de administración de usuarios y de edición o eliminación de exámenes, a la visualización y descarga en el rol paciente, y a requisitos que exigen entorno productivo, monitoreo o pruebas con usuarios; se declaran como brecha de la iteración y no se presentan como probados. El caso más relevante es NFR-SEG-9: el login funciona (CP005) pero la revocación no (CP006), y por eso el veredicto es GO CONDICIONADO.

Los requisitos con cobertura formal más débil (NFR-PERF-2 y NFR-PERF-3) dependen del retardo intencional de 5 s (H-05) y de un entorno de carga que no existe en el equipo; por eso se medirán con JMeter en la próxima iteración, junto con la re-ejecución de CP005/CP006 y del análisis estático una vez corregidos H-01 a H-04.

4.2 Casos de prueba documentados

ID	CP001
NOMBRE	Registro de paciente con datos de formato válido
REQUERIMIENTO	RQ-07 (RF-2)
DESCRIPCIÓN	Demostrar que un médico puede registrar un paciente con datos que cumplen el formato y que el sistema lo guarda y lo lista. El RUT 12.345.678-9 es válido en formato, pero su dígito verificador real es 5: el caso verifica la validación de formato, no la identidad del paciente (ver H-03).
PRECONDICIONES	Docker levantado (backend 4444, frontend 3333). Sesión iniciada como médico (token válido). RUT y email no registrados previamente.
PASOS	1. Ir a Pacientes > Crear (o POST /pacientes). 2. Ingresar RUT 12.345.678-9, primer nombre Juan, apellido Pérez, email juan@test.cl, fecha 2000-01-01, sexo M. 3. Guardar.
RESULTADO ESPERADO	El sistema crea el paciente con 201, lo muestra en el listado y genera registro de auditoría PACIENTES_CREAR.
CRITERIO DE ACEPTACIÓN	Respuesta 201; fila visible en Gestión de Pacientes; sin mensajes de validación. La auditoría de la creación se verificó por revisión del código (insert en pacientes.js), no en esta captura.
DATOS DE PRUEBA	RUT 12.345.678-9; Juan Pérez; juan@test.cl; 2000-01-01; sexo M.
TIPO DE PRUEBA	Funcional (positivo)

ID	CP002
NOMBRE	Email sin formato válido
REQUERIMIENTO	RQ-07 (RF-2)
DESCRIPCIÓN	Demostrar que el sistema rechaza un registro con email inválido (sin @) y no guarda datos.
PRECONDICIONES	Docker levantado. Sesión iniciada como médico.
PASOS	1. Ir a Pacientes > Crear. 2. Ingresar RUT 9.876.543-2 (válido), nombre Juan, apellido Pérez, email "correosinarroba" (sin @), fecha 2000-01-01, sexo M. 3. Guardar.
RESULTADO ESPERADO	El sistema responde 400 con detalle en el campo email ("El email debe tener un formato válido") y no guarda el paciente.
CRITERIO DE ACEPTACIÓN	Respuesta 400; mensaje de validación visible en el formulario; la lista de pacientes no agrega filas.

DATOS DE PRUEBA	RUT 9.876.543-2; email correosinarroba.
TIPO DE PRUEBA	Funcional (negativo)

ID	CP003
NOMBRE	Registro duplicado (RUT y email existentes)
REQUERIMIENTO	RQ-07 (RF-2)
DESCRIPCIÓN	Demostrar que el sistema rechaza el registro de un paciente cuyo RUT y email ya existen.
PRECONDICIONES	Docker levantado. Sesión iniciada como médico. RUT y email del CP001 ya registrados.
PASOS	1. Ir a Pacientes > Crear. 2. Ingresar el mismo RUT y email del CP001. 3. Guardar.
RESULTADO ESPERADO	El sistema responde 400 indicando que el RUT o email ya existe, sin agregar filas al listado.
CRITERIO DE ACEPTACIÓN	Respuesta 400 por duplicado; el listado mantiene la misma cantidad de filas.
DATOS DE PRUEBA	RUT 12.345.678-9 y juan@test.cl (del CP001).
TIPO DE PRUEBA	Funcional (negativo)

ID	CP004
NOMBRE	Borde: nombre de 1 letra (mínimo 2)
REQUERIMIENTO	RQ-07 (RF-2)
DESCRIPCIÓN	Demostrar que el sistema valida el largo mínimo del primer nombre (2 caracteres) en el borde inferior.
PRECONDICIONES	Docker levantado. Sesión iniciada como médico.
PASOS	1. Ir a Pacientes > Crear. 2. Ingresar RUT 19.123.456-7, primer nombre "A" (1 letra), apellido Pérez, email juan@test.cl, fecha 2000-01-01, sexo F. 3. Guardar.
RESULTADO ESPERADO	El sistema responde 400 con detalle en primer_nombre (mínimo 2) y no guarda el paciente.
CRITERIO DE ACEPTACIÓN	Respuesta 400; campo marcado con detalle específico; no se crea la fila.

DATOS DE PRUEBA	Primer nombre "A" (borde inferior); RUT 19.123.456-7; juan@test.cl; sexo F.
TIPO DE PRUEBA	Funcional (borde)

ID	CP005
NOMBRE	Autenticación exitosa y uso del token
REQUERIMIENTO	RQ-01 (RF-1.1, NFR-SEG-9)
DESCRIPCIÓN	Demostrar que un usuario registrado puede autenticarse y que el token JWT emitido es válido para consumir la API.
PRECONDICIONES	Docker levantado. Usuario médico registrado con credenciales conocidas.
PASOS	1. Enviar POST /autenticacion/login con usuario y contraseña del usuario de prueba. 2. Usar el token recibido en el header Authorization para GET /pacientes.
RESULTADO ESPERADO	El login responde 200 con token JWT que incluye el rol; el listado de pacientes responde 200 con el token.
CRITERIO DE ACEPTACIÓN	Respuesta 200 en ambas llamadas; token decodificable con rol correcto; auditoría de login generada.
DATOS DE PRUEBA	Usuario médico de prueba (credenciales del entorno Docker).
TIPO DE PRUEBA	Seguridad (positivo)

ID	CP006
NOMBRE	Reutilización del token tras cerrar sesión
REQUERIMIENTO	RQ-01 (RF-1.2, NFR-SEG-9)
DESCRIPCIÓN	Demostrar que tras cerrar sesión el token queda revocado y no puede reutilizarse.
PRECONDICIONES	Docker levantado. Sesión iniciada con token válido emitido por login.
PASOS	1. Login válido y obtención del token. 2. Cerrar sesión desde la interfaz. 3. Reintentar el token previo contra un endpoint protegido con Postman.
RESULTADO ESPERADO	El sistema responde 401 Unauthorized: el token ya no es válido tras el cierre de sesión.
CRITERIO DE ACEPTACIÓN	Respuesta 401 al reusar el token; ningún endpoint acepta la sesión revocada.
DATOS DE PRUEBA	Token JWT emitido en el paso 1.

TIPO DE PRUEBA	Seguridad (negativo)

ID	CP007
NOMBRE	Paciente intenta editar un examen (RBAC)
REQUERIMIENTO	RQ-02 (RB-1, NFR-SEG-4)
DESCRIPCIÓN	Demostrar que el control de acceso por rol impide a una paciente modificar un examen médico.
PRECONDICIONES	Docker levantado. Usuario paciente registrado. Examen existente con ID conocido.
PASOS	1. Login como paciente. 2. Enviar PUT /examenes/{id} con datos modificados usando el token de paciente. 3. Verificar la base de datos.
RESULTADO ESPERADO	El sistema responde 403 Forbidden y la base de datos no registra cambios en el examen.
CRITERIO DE ACEPTACIÓN	Respuesta 403; verificación directa en PostgreSQL: sin cambios en la fila del examen.
DATOS DE PRUEBA	Token de rol paciente; PUT con datos de examen modificados.
TIPO DE PRUEBA	Seguridad / RBAC

ID	CP008
NOMBRE	Carga y descarga de documento (integridad)
REQUERIMIENTO	RQ-03 (RF-4.2, RF-4.3)
DESCRIPCIÓN	Demostrar que un documento de examen se carga y descarga íntegro dentro del tiempo exigido.
PRECONDICIONES	Docker levantado. Sesión como médico. Examen existente. Archivo de prueba disponible (formato permitido, bajo 10 MB).
PASOS	1. Adjuntar el documento al examen (subida). 2. Entrar a Mis exámenes y descargar el documento. 3. Calcular SHA256 del archivo original y del descargado.
RESULTADO ESPERADO	Subida 201 y descarga 200 con el archivo íntegro y legible en menos de 10 segundos (considerando el retardo de 5 s de los endpoints).
CRITERIO DE ACEPTACIÓN	Hash SHA256 idéntico entre original y descarga; tiempo total bajo 10 s; archivo legible.
DATOS DE PRUEBA	Documento PDF de prueba; hash SHA256 del original.

TIPO DE PRUEBA	Funcional / integridad

5. Ejecución de pruebas

5.1 Resumen de ejecución

Se ejecutó la totalidad del plan de pruebas semestral en el entorno Docker del caso: los 8 casos de prueba diseñados fueron ejecutados (4 el 06-10-2026 y 4 el 16-10-2026), con 7 aprobados y 1 fallido (CP006). El fallo corresponde a un defecto real del sistema (H-02: no existe revocación de sesión), no a un error de la prueba. Adicionalmente se ejecutó el análisis estático automatizado con SonarQube Community 9.9.8 sobre las 10.133 líneas de código del repositorio, con Quality Gate OK, 73 issues (1 bug, 72 code smells de los cuales 8 críticos, 0 vulnerabilidades), 3 security hotspots, 14,5% de duplicación, deuda técnica de 465 minutos y 0% de cobertura por ausencia de reportes LCOV.

El Quality Gate reportó estado OK, pero la respuesta del servidor muestra la lista de condiciones vacía ("conditions": []): el gate no tenía umbrales configurados, por lo que el estado certifica que el análisis se completó, no que se hayan superado criterios de calidad. Por eso no se usa como criterio de aceptación cumplido, sino como evidencia de ejecución del SAST.

Diseñados	Ejecutados	Aprobados (Pass)	Fallidos (Fail)	Bloqueados	Pendientes
8	8	7	1	0	0

5.2 Detalle de ejecución de casos

Detalle caso a caso. Ejecutor de todos los casos: Benjamín Ibáñez. Evidencias completas en Mis_Entregables/U2.3_Ejecucion/Evidencias (capturas de pantalla para CP001–CP004; respuestas de API en JSON/TXT/PDF para CP005–CP008; registros SonarQube). Los incidentes por caso se detallan en la sección 5.3 y en la nota de ejecución que sigue a la tabla.

Caso	Descripción	Resultado	Evidencia	Tipo de prueba	Incidente	Fecha y responsable
CP001	Registro válido de paciente (201, aparece en listado, deja auditoría)	Pass	Grupo1_CP001_Front_201_Pass.png	Funcional (positivo)	—	06-10-2026 · B. Ibáñez
CP002	Email sin @ (400 con detalle en email, no guarda)	Pass	Grupo1_CP002_Front_tooltip_Pass.png; Grupo1_CP002_Backend_400_Pass.png	Funcional (negativo)	—	06-10-2026 · B. Ibáñez

Caso	Descripción	Resultado	Evidencia	Tipo de prueba	Incidente	Fecha y responsable
CP003	Registro duplicado: mismo RUT y email del CP001 (400, sin fila nueva)	Pass	Grupo1_CP003_Duplicado_400_Pass.png	Funcional (negativo)	—	06-10-2026 · B. Ibáñez
CP004	Borde: primer nombre de 1 letra, mínimo 2 (400 con detalle)	Pass	Grupo1_CP004_Front_borde_Pass.png; Grupo1_CP004_Backend_400_Pass.png	Funcional (borde)	—	06-10-2026 · B. Ibáñez
CP005	Login con credenciales válidas: token JWT con rol y uso del token en la API	Pass	Grupo1_CP005_Login_200.json; Grupo1_CP005_Uso_token_lista_pacientes_200.json	Seguridad (positivo)	—	16-10-2026 · B. Ibáñez
CP006	Reutilización del token tras cerrar sesión (revocación)	Fail	Grupo1_CP006_Reuso_token_tras_logout_200.json; Grupo1_CP006_Busqueda_logout.txt	Seguridad (negativo)	H-02 (Grave): el token sigue válido tras el logout; esperado 401, obtenido 200.	16-10-2026 · B. Ibáñez
CP007	Rol paciente intenta editar un examen (esperado 403; 5.036 ms)	Pass	Grupo1_CP007_PUT_403.json; Grupo1_CP007_Verificacion_BD_sin_cambios.txt	Seguridad / RBAC	—	16-10-2026 · B. Ibáñez
CP008	Carga y descarga de documento (subida 5.050 ms, descarga 37 ms, SHA256 idéntico)	Pass	Grupo1_CP008_Subida_documento_201.json; Grupo1_CP008_Descarga_documento_200.pdf; Grupo1_CP008_Integridad_y_tiempo.txt	Funcional / integridad	—	16-10-2026 · B. Ibáñez

Nota de ejecución: los valores efectivamente usados en la corrida difieren en identidad de los diseñados en la sección 4, pero pertenecen a la misma clase de equivalencia. CP001 se ejecutó con el paciente que muestra la captura (RUT 21.437.351-4, fecha 31-12-1999); CP002 usó el RUT 9.876.543-2 con nombre «Pedro» y apellido «Soto»; CP003 intentó el RUT 21.437.351-3, ya registrado; y CP004 usó el RUT 7.654.321-0 con primer nombre «A». Las capturas del Anexo A muestran esos valores. Incidentes: CP006 registra H-02; los demás casos no generaron incidentes.

5.3 Criticidad de incidentes

La escala de criticidad se definió según el impacto real en el negocio del caso (sistema de salud): un problema que expone datos de salud o permite acceso indebido es crítico; una función importante afectada es grave; una función secundaria o degradación de experiencia es moderada; un defecto de apariencia o de divulgación menor es leve.

Nivel	Descripción	Impacto	Acción
CRÍTICO	El defecto impide completamente el funcionamiento del sistema o de una funcionalidad esencial para el negocio; bloquea el flujo principal.	Muy alto: el sistema no puede ser usado	Requiere remediación inmediata
GRAVE	El defecto afecta funcionalidades importantes, pero el sistema aún funciona parcialmente; existe workaround pero la experiencia se ve comprometida.	Alto: afecta áreas principales del usuario	Debe corregirse con urgencia
MODERADO	El defecto afecta funcionalidades secundarias o casos específicos sin detener el flujo principal.	Medio: el usuario puede trabajar con molestias o limitaciones	Se corrige en próxima iteración
LEVE	Defecto de apariencia, estética o texto que no afecta la funcionalidad.	Bajo: no impacta la operación	Se corrige según disponibilidad

Hallazgos identificados. La revisión manual del código guiada por OWASP Top 10 y la ejecución de los casos identificaron 7 hallazgos: 1 crítico, 3 graves, 2 moderados y 1 leve, concentrados en credenciales, sesiones y validación de identidad:

ID	Hallazgo	Severidad	Evidencia	Impacto	Acción recomendada
H-01	Contraseñas almacenadas y comparadas en texto plano	Crítico	auth.js:21 (comparación directa) y :110 (inserción sin hash)	Fuga de credenciales de médicos, pacientes y administradores; acceso indebido a datos clínicos. Incumple Ley 19.628 y OWASP A02.	Hashear con bcrypt/argon2 y sal; migrar credenciales existentes; re-certificar el login.

ID	Hallazgo	Severidad	Evidencia	Impacto	Acción recomendada
H-02	Sesiones JWT sin revocación y secreto por defecto inseguro	Grave	auth.js:8 y :44-55 (token 2 h sin logout); docker-compose.yml:35. Confirmado en vivo por CP006 (token reutilizado respondió 200)	Una sesión robada sigue válida hasta 2 horas sin poder revocarse; sin secreto definido, los tokens podrían forjarse (OWASP A07).	Implementar logout/revocación; exigir secreto fuerte por entorno (fallar si no existe); acortar expiración y evaluar refresh tokens.
H-03	RUT validado solo por formato, sin dígito verificador	Grave	validations/pacientes.js:8 y utils/rut.js:5-8. Confirmado en CP001: se aceptó con 201 el RUT 21.437.351-4, cuyo dígito verificador correcto es 3 (Anexo A)	Fichas clínicas asociadas a RUT inexistentes; riesgo de incumplir la identificación del paciente (Ley 20.584).	Validar dígito verificador (módulo 11); rechazar RUT inválidos; revisar registros existentes.
H-04	Directorio de documentos servido como estático sin autenticación (riesgo latente)	Grave (latente)	app.js:43 (express.static sin autenticación). Verificado: uploads sin archivos hoy (los documentos van a la BD)	Si en el futuro se almacena en disco, cualquier persona con la URL podría descargar documentos clínicos sin token, saltándose el RBAC (OWASP A01).	Eliminar el montaje estático o protegerlo con autenticación; usar solo el endpoint /documentos/:id con control de acceso.
H-05	Retardo intencional de 5 s en endpoints de listado/creación	Moderado	middleware/delay.js:5 aplicado en pacientes.js:145, examenes.js y documentos.js. Confirmado: CP007 5.036 ms; CP008 5.050 ms	Los listados y operaciones demoran ~5 s; degrada la experiencia e impide la meta de <300 ms (NFR-PERF-1).	Retirar el delay en entornos reales; medir tiempos y definir metas de desempeño.
H-06	CORS abierto sin restricción de origen	Moderado	app.js:21 (CORS sin restricción)	Permite peticiones desde cualquier origen; combinado con un XSS facilitaría el robo del token (OWASP A05).	Restringir CORS al origen del frontend (FRONTEND_URL).

ID	Hallazgo	Severidad	Evidencia	Impacto	Acción recomendada
H-07	Mensajes de error internos expuestos al cliente	Leve	Patrón err.message en auth.js:67, documentos.js:115 y otras rutas	Los errores internos entregan información útil para un atacante (OWASP A05).	Responder mensajes genéricos; conservar el detalle solo en los logs del servidor.

Fortalezas observadas (evidencia positiva): control de acceso por roles verificado en código y en vivo (401/403, CP007 sin cambios en la base de datos); auditoría insertada en login, registro y operaciones CRUD; validaciones de formulario completas con mensajes por campo; carga de archivos controlada (límite 10 MB, tipos PDF/JPG/PNG); y consultas SQL parametrizadas en las rutas revisadas, que reducen el riesgo de inyección SQL.

5.4 Análisis estático con SonarQube: incidencias priorizadas

Como parte de la estrategia SAST se ejecutó SonarQube Community 9.9.8 sobre el repositorio completo (10.133 líneas de código, análisis del 16-10-2026). La exportación oficial reporta 73 incidencias en total: 8 críticas, 32 mayores, 32 menores y 1 de nivel informativo. El detalle íntegro está disponible en la evidencia Grupo1_SonarQube_issues.json; el registro de métricas, en Grupo1_SonarQube_measures.json.

De acuerdo con la consigna de la unidad, se documentan aquí en detalle las 25 incidencias de mayor prioridad. El criterio de selección fue: (1) todas las incidencias de severidad crítica (8, regla S3776, complejidad cognitiva); (2) 17 incidencias de severidad mayor elegidas por relevancia funcional y diversidad de reglas, priorizando el módulo propio del equipo (pacientes), el backend y los componentes de autenticación/autorización. Cada ficha del Anexo B incluye la explicación en español, el fragmento de código real señalado por el escáner y, para las críticas, el registro JSON de la incidencia tal como lo entregó SonarQube.

La tabla siguiente resume el conjunto priorizado (SQ-01 a SQ-25); las descripciones están en español.

ID	Severidad	Regla	Archivo:línea	Incidencia (resumen)
SQ-01	Crítica	Complejidad cognitiva (S3776)	auth.js:71	Complejidad cognitiva de 17 (umbral 15)
SQ-02	Crítica	Complejidad cognitiva (S3776)	pacientes.js:48	Complejidad cognitiva de 22 (umbral 15)
SQ-03	Crítica	Complejidad cognitiva (S3776)	pacientes.js:276	Complejidad cognitiva de 21 (umbral 15)
SQ-04	Crítica	Complejidad cognitiva (S3776)	users.js:283	Complejidad cognitiva de 27 (umbral 15)
SQ-05	Crítica	Complejidad cognitiva (S3776)	users.js:437	Complejidad cognitiva de 24 (umbral 15)
SQ-06	Crítica	Complejidad cognitiva (S3776)	ExamenForm.jsx:47	Complejidad cognitiva de 33 (umbral 15)
SQ-07	Crítica	Complejidad cognitiva (S3776)	PacienteForm.jsx:21	Complejidad cognitiva de 22 (umbral 15)
SQ-08	Crítica	Complejidad cognitiva (S3776)	UsuarioForm.jsx:173	Complejidad cognitiva de 39 (umbral 15)
SQ-09	Mayor	Asignación redundante (S4165)	documentos.js:244	Asignación redundante a "contentType"
SQ-10	Mayor	Template literals anidados (S4624)	users.js:60	Template literal anidado
SQ-11	Mayor	Componente definido en render (S6478)	PacientesTabList.jsx:53	Componente definido durante el render
SQ-12	Mayor	Componente definido en render (S6478)	PacientesTabList.jsx:86	Componente definido durante el render
SQ-13	Mayor	Componente definido en render (S6478)	PacientesTabList.jsx:97	Componente definido durante el render

SQ-14	Mayor	Componente definido en render (S6478)	PacientesTabList.jsx:122	Componente definido durante el render
SQ-15	Mayor	Componente definido en render (S6478)	AuditoriaList.jsx:51	Componente definido durante el render
SQ-16	Mayor	Componente definido en render (S6478)	ExamenesTabList.jsx:71	Componente definido durante el render
SQ-17	Mayor	Componente definido en render (S6478)	UsuariosTabList.jsx:53	Componente definido durante el render
SQ-18	Mayor	Ternario anidado (S3358)	ExamenForm.jsx:451	Ternario anidado
SQ-19	Mayor	Ternario anidado (S3358)	PacientesTabCreate.jsx:501	Ternario anidado
SQ-20	Mayor	Ternario anidado (S3358)	UsuariosTabCreate.jsx:188	Ternario anidado
SQ-21	Mayor	Índice como key (S6479)	Ayuda.jsx:119	Índice del array usado como key
SQ-22	Mayor	Asignación muerta (S1854)	RoleGuard.jsx:28	Variable "location" asignada y nunca leída
SQ-23	Mayor	Asignación muerta (S1854)	PacienteForm.jsx:12	Variable "loading" asignada y nunca leída
SQ-24	Mayor	Asignación muerta (S1854)	PacientesTabList.jsx:24	Variable "role" asignada y nunca leída
SQ-25	Mayor	Fuentes sin genérica (S4649)	index.css:6	Pila de fuentes sin tipografía genérica

Las fichas completas de estas 25 incidencias (explicación, código real y JSON de las críticas) se presentan en el Anexo B. El registro se actualizará con cada análisis estático; la baja de una incidencia solo procede con corrección de código y re-escaneo, no por decisión administrativa.

6. Análisis ético y normativo de los resultados

6.1 Legal y normativo (contexto chileno)

- Ley 19.628 sobre protección de la vida privada: los datos de salud son datos sensibles y el sistema debe resguardar su confidencialidad. Los hallazgos H-01, H-02 y H-04 afectan directamente este deber: credenciales en texto plano, sesiones no revocables y un directorio de archivos sin autenticación constituyen riesgos concretos de acceso indebido a datos personales de salud.
- Ley 20.584 sobre derechos y deberes del paciente: la ficha clínica es reservada y su acceso debe estar controlado y trazado. La auditoría existente es una fortaleza alineada con este deber; sin embargo, la falta de revocación de sesión (H-02) debilita el control de acceso y la validación incompleta del RUT (H-03) compromete la correcta identificación del paciente frente a su ficha.
- ISO/IEC 25010: adecuación funcional conforme en los flujos probados (CP001–CP004, CP007 y CP008); eficiencia de desempeño limitada por el retardo de 5 s (H-05,

confirmado en CP007/CP008); seguridad y confiabilidad comprometidas por H-01 a H-04; mantenibilidad con oportunidades de mejora (8 issues críticos de complejidad S3776, deuda técnica de 465 minutos y 14,5% de duplicación según SonarQube).

- OWASP Top 10 (2021): los hallazgos se mapearon a A01 (control de acceso roto — H-04), A02 (fallos criptográficos — H-01 y H-02), A05 (configuración de seguridad incorrecta — H-06 y H-07) y A07 (fallos de identificación y autenticación — H-02 y H-03). El análisis estático no detectó vulnerabilidades catalogadas (0 en SonarQube), lo que confirma que los riesgos reales residen en la lógica de negocio y la configuración, no en patrones detectables automáticamente: de ahí la necesidad de combinar SAST con revisión manual experta.

6.2 Ética

- Se registran todos los hallazgos encontrados, sin ocultar ni minimizar riesgos sobre datos de salud, y cada uno se sustenta en evidencia (archivo y línea de código o ejecución documentada).
- No se inventaron resultados: lo no ejecutado (DAST con OWASP ZAP, pruebas de carga formales, pruebas de usabilidad con usuarios) se declara como pendiente para la siguiente iteración.
- Los datos utilizados en las pruebas fueron ficticios y no se manipuló ni corrigió el código fuente fuera de las reglas de la actividad: los defectos se registran y se recomienda su corrección.
- Recomendación responsable: no liberar a producción un sistema de salud con riesgos conocidos de confidencialidad (credenciales en texto plano y sesiones no revocables) sin corregirlos primero. Informar al cliente los riesgos residuales es una obligación ética del equipo de aseguramiento.

7. Conclusiones, recomendaciones y acciones pendientes

El sistema Cuidarte+ cumple funcionalmente en los flujos probados (registro de pacientes con validaciones, control de acceso por roles y gestión de documentos con integridad verificada) y presenta fortalezas de control de acceso y auditoría. El análisis estático con SonarQube no detectó vulnerabilidades catalogadas ni bugs de seguridad, lo que coincide con la revisión manual: los riesgos están en la lógica de negocio y la configuración, no en patrones que el análisis estático detecte automáticamente; en cambio registra 8 issues críticos de complejidad cognitiva y 14,5% de duplicación (deuda técnica de 465 minutos).

Un sistema que administra datos de salud no puede liberarse con credenciales en texto plano (H-01), sesiones no revocables confirmadas en ejecución (H-02, CP006), validación de identidad incompleta (H-03) y un directorio de archivos sin autenticación (H-04). Con toda la evidencia reunida, el veredicto del equipo es:

Veredicto: GO CONDICIONADO. No existe una falla funcional bloqueante en los flujos probados, pero los hallazgos H-01 a H-04 deben corregirse obligatoriamente antes de liberar el sistema a producción. Si estas condiciones no se cumplen, el veredicto debe escalar a NO-GO.

Acciones pendientes y recomendaciones

- Corregir las condiciones obligatorias (H-01 a H-04) y re-ejecutar CP005/CP006 y el análisis SonarQube para evidenciar la mejora (revocación efectiva del token, reducción de issues críticos y deuda técnica).
- Revisar los 8 issues críticos de complejidad cognitiva y los 3 security hotspots señalados por SonarQube para priorizar la refactorización.
- Retirar el retardo de 5 s (H-05) y medir formalmente los tiempos de los CRUD contra las metas del ERS (<300 ms en el 95%, 200 usuarios concurrentes) con JMeter.
- Ejecutar pruebas dinámicas (DAST) con OWASP ZAP y pruebas de usabilidad con usuarios, incluyendo adultos mayores (NFR-USAB), en la siguiente iteración.
- Re-certificar el sistema después de aplicar las correcciones obligatorias y actualizar el veredicto (GO/NO-GO final).

Referencias del trabajo previo: Plan Maestro de Pruebas Unidad 2 (versión fusionada, 16-10-2026), Guía de Trabajo 2.2 (diseño de casos) y Bitácora de Ejecución 2.3 con sus evidencias, en Mis_Entregables/. Cada integrante del grupo puede explicar y defender la totalidad de este trabajo.

Anexo A. Evidencias de ejecución

Las capturas reproducen el entorno Docker del caso (frontend http://localhost:3333, backend http://localhost:4444) y los archivos archivados en Mis_Entregables/U2.3_Ejecucion/Evidencias/. Se incluyen para que la evidencia sea legible en el propio informe, conforme a los aspectos formales del encargo.

A.1 Casos CP001–CP004 (capturas de interfaz y API)

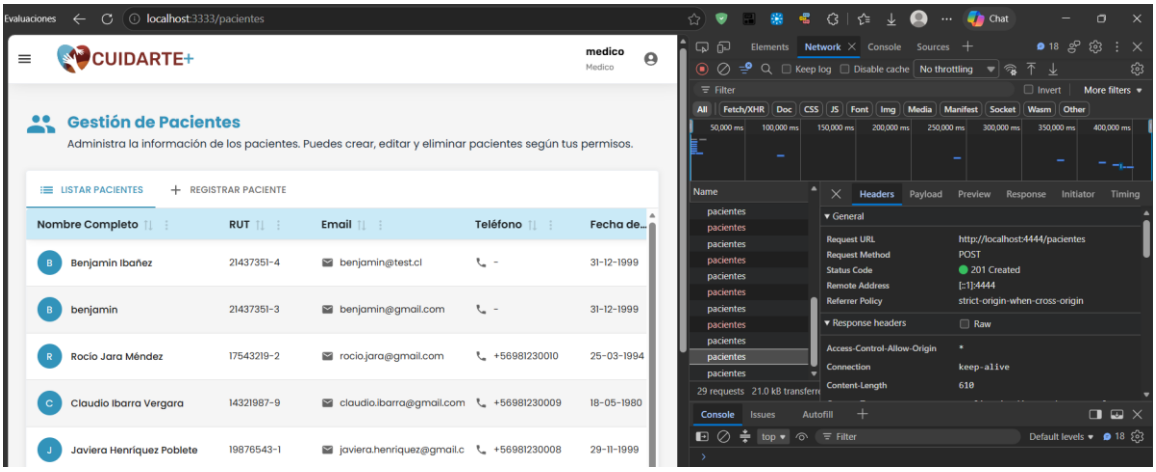


Figura A.1. CP001: POST /pacientes responde 201 y el paciente aparece en Gestión de Pacientes. Archivo: Grupo1_CP001_Front_201_Pass.png.

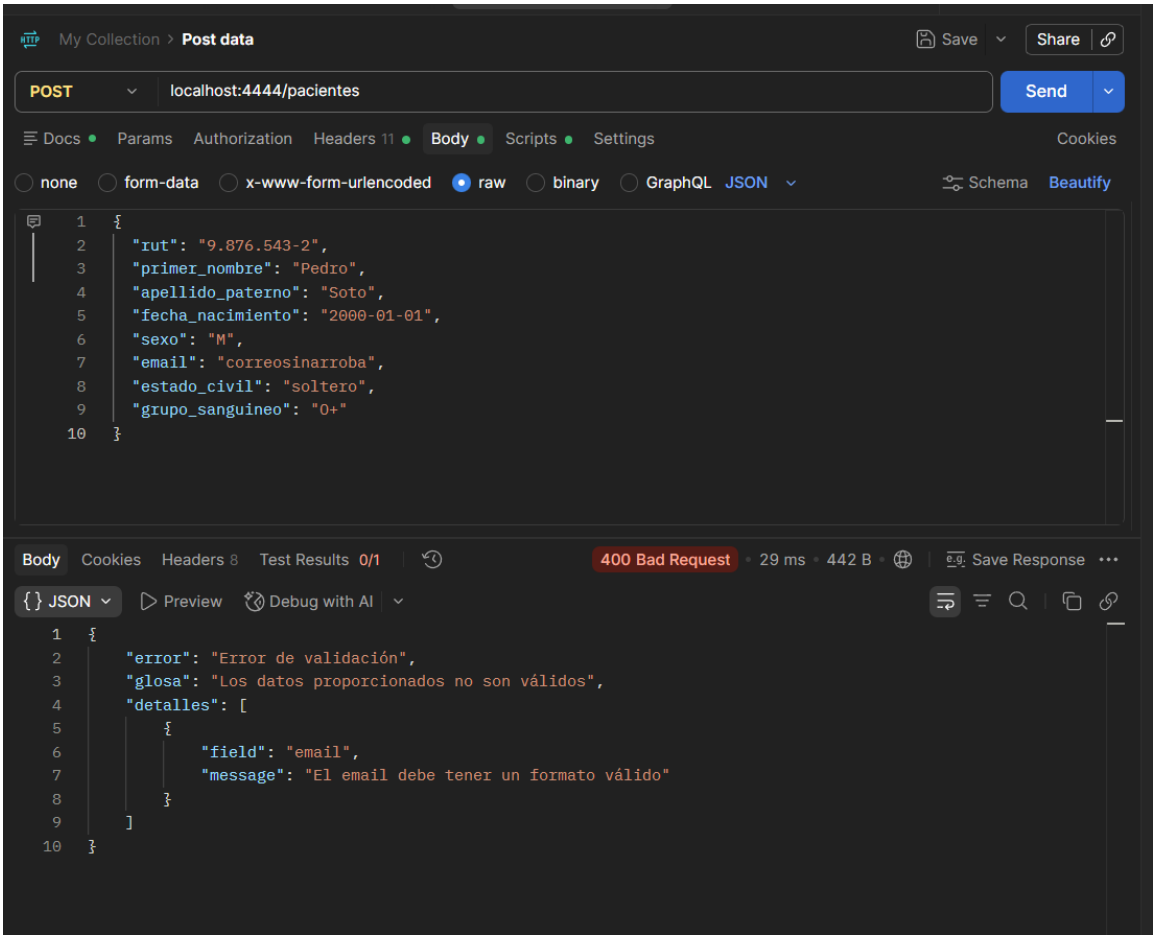


Figura A.2. CP002: email sin @ rechazado con 400 y detalle del campo email. Archivo: Grupo1_CP002_Backend_400_Pass.png.

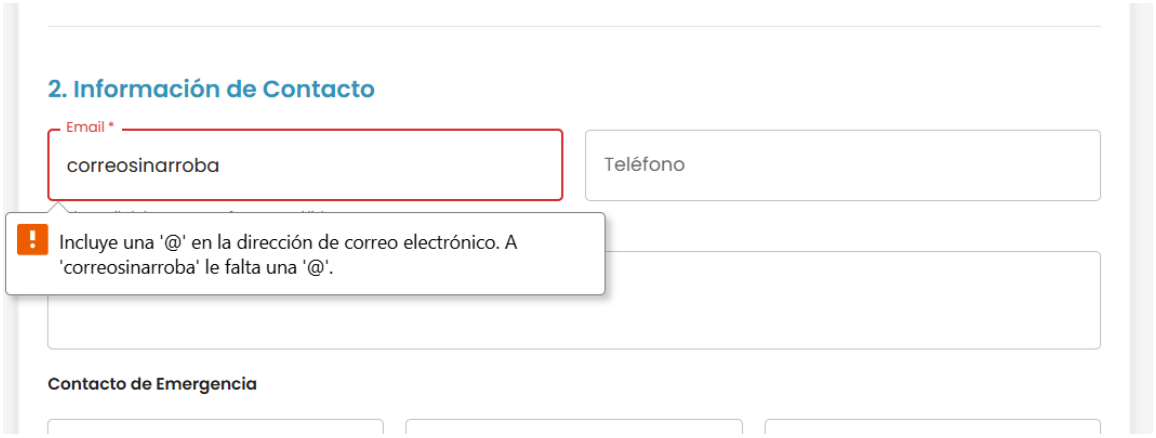


Figura A.3. CP002: el formulario marca el campo email con el mensaje de validación. Archivo: Grupo1_CP002_Front_tooltip_Pass.png.

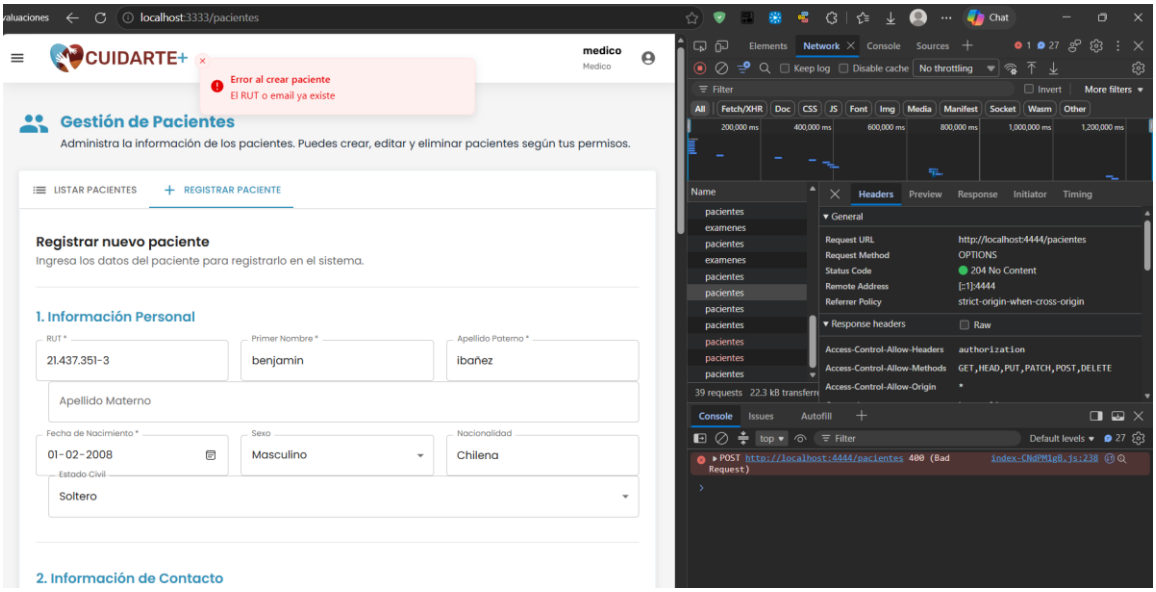


Figura A.4. CP003: RUT ya registrado rechazado con 400, sin agregar filas. Archivo: Grupo1_CP003_Duplicado_400_Pass.png.

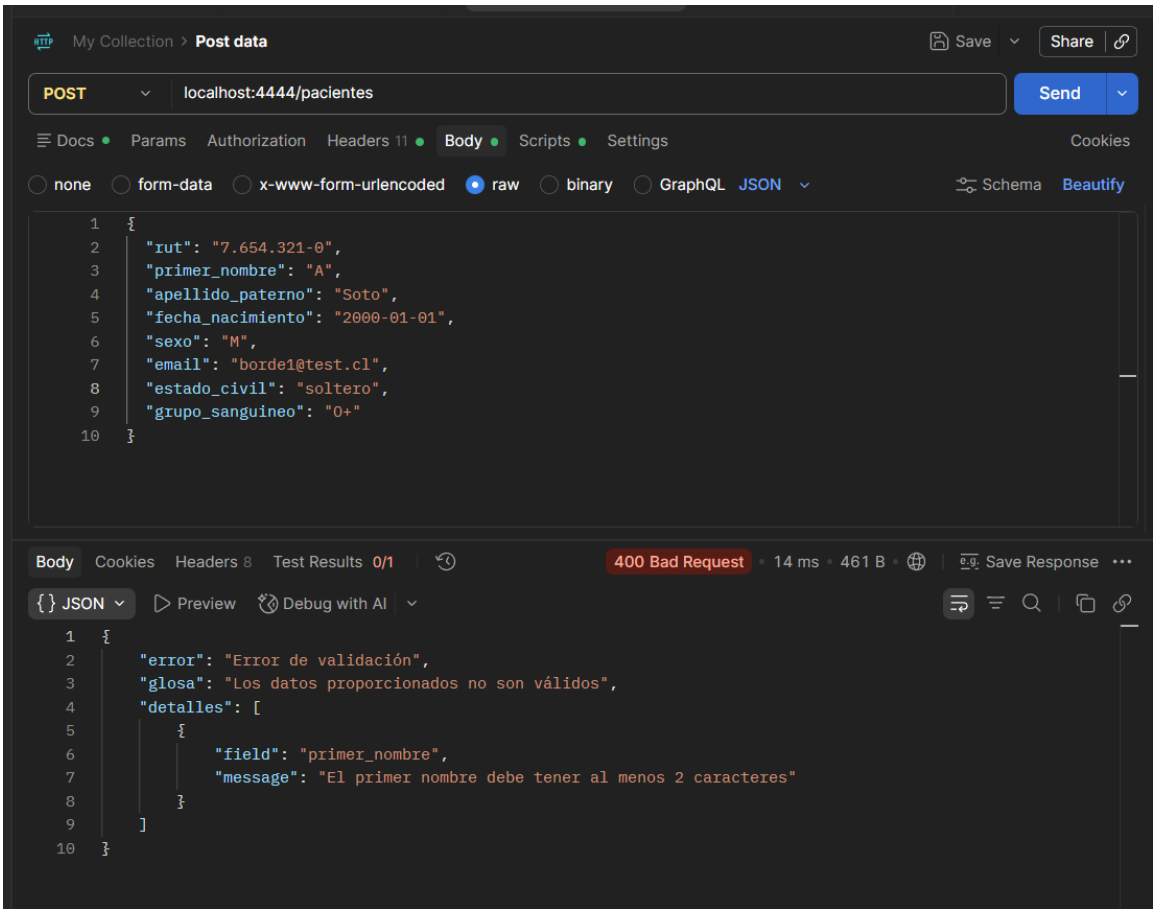


Figura A.5. CP004: primer nombre de 1 letra rechazado con 400 en primer_nombre. Archivo: Grupo1_CP004_Backend_400_Pass.png.

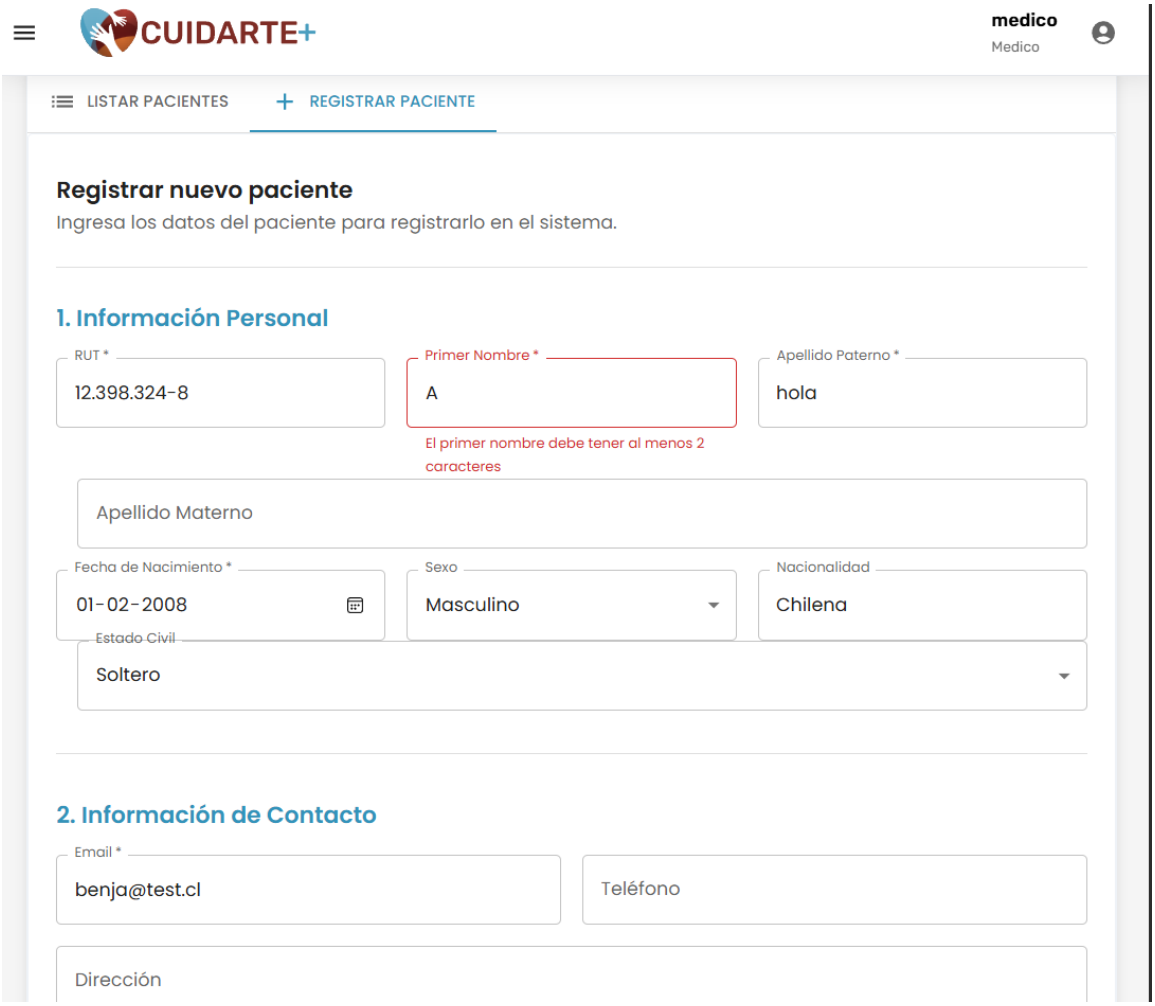


Figura A.6. CP004: el formulario marca el borde inferior con el mínimo de 2 caracteres. Archivo: Grupo1_CP004_Front_borde_Pass.png.

A.2 Casos CP005–CP008 (respuestas de API)

Caso	Petición	Respuesta observada	Esperado / veredicto
CP005	POST /autenticacion/login (médico) y GET /pacientes con el token	200 con token que incluye el rol; GET 200	Pass (esperado 200)
CP006	GET /pacientes con el token obtenido antes del logout	200: el token sigue vigente	Fail (esperado 401; H-02)
CP007	PUT /examenes/{id} con token de paciente	403 y verificación en PostgreSQL sin cambios (5.036 ms)	Pass (esperado 403)
CP008	POST /documentos y GET /documentos/{id}	201; descarga 200 con SHA256 idéntico (5.050 ms / 37 ms)	Pass (esperado 201/200)

Los archivos completos (JSON, TXT y PDF) y los registros de SonarQube están en la carpeta de evidencias citada en la sección 5.2; esta tabla resume la respuesta observada de cada caso y su veredicto.

Anexo B. Fichas de las 25 incidencias priorizadas de SonarQube

Fuente: exportación oficial de SonarQube Community 9.9.8 del 16-10-2026 (Grupo1_SonarQube_issues.json, 73 incidencias) más el código fuente vigente del repositorio a la misma fecha. Cada ficha indica: identificador, severidad (traducida del inglés: Critical = crítica, Major = mayor), explicación en español, fragmento del código real señalado por el escáner (la línea de anclaje va marcada con >>) y, para las incidencias críticas, el registro JSON correspondiente de la exportación.

Las incidencias se ordenan por severidad decreciente: SQ-01 a SQ-08 críticas; SQ-09 a SQ-25 mayores.

SQ-01 · Crítica · BACKEND/src/routes/auth.js:71

Regla javascript:S3776 — Se detecta complejidad cognitiva excesiva de la función (umbral permitido: 15). Las funciones que acumulan demasiadas decisiones encadenadas son difíciles de mantener y de probar. Recomendación: Extraer la lógica a funciones auxiliares de un solo propósito; además facilita las pruebas unitarias.

Fragmento de código real (BACKEND/src/routes/auth.js, líneas 68–77):

```
     68    }
     69  });
     70
>>   71  router.post("/registro", validate(registerSchema), async (req, res) => {
     72    const { nombre_usuario, password, correo, rut } = req.body;
     73    try {
     74      // Normalizar RUT antes de procesar
     75      const normalizedRut = rut ? normalizeRut(rut) : null;
     76
     77      // Verificar si el RUT ya existe en pacientes
```

Registro JSON de la exportación oficial (extracto de Grupo1_SonarQube_issues.json):


```
{
  "key": "AaFGL_QHh7pI0-4QFeO7",
  "rule": "javascript:S3776",
  "severity": "CRITICAL",
  "component": "cuidarte-plus:BACKEND/src/routes/auth.js",
  "project": "cuidarte-plus",
  "line": 71,
  "message": "Refactor this function to reduce its Cognitive Complexity from 17 to the 15 allowed.",
  "effort": "7min",
  "debt": "7min",
  "type": "CODE_SMELL",
```

```
  "creationDate": "2026-09-29T21:46:30+0000"
}
```

SQ-02 · Crítica · BACKEND/src/routes/pacientes.js:48

Regla javascript:S3776 — Se detecta complejidad cognitiva excesiva de la función (umbral permitido: 15). Las funciones que acumulan demasiadas decisiones encadenadas son difíciles de mantener y de probar. Recomendación: Extraer la lógica a funciones auxiliares de un solo propósito; además facilita las pruebas unitarias.

Fragmento de código real (BACKEND/src/routes/pacientes.js, líneas 45–54):

```
    45  }
    46
    47  // Crear paciente (solo médico y admin)
>>  48  router.post("/", validate(pacienteSchema), async (req, res) => {
    49    try {
    50      const authCheck = await requireRole(req, ["medico", "admin"]);
    51      if (authCheck.error) {
    52        return res.status(authCheck.status).json({
    53          error: authCheck.error,
    54          glosa: "No tiene permisos para realizar esta acción",
```

Registro JSON de la exportación oficial (extracto de Grupo1_SonarQube_issues.json):

```
{
  "key": "AaFGL_Pwh7pI0-4QFeO3",
  "rule": "javascript:S3776",
  "severity": "CRITICAL",
  "component": "cuidarte-plus:BACKEND/src/routes/pacientes.js",
  "project": "cuidarte-plus",
  "line": 48,
  "message": "Refactor this function to reduce its Cognitive Complexity from 22 to the 15 allowed.",
  "effort": "12min",
  "debt": "12min",
  "type": "CODE_SMELL",
  "creationDate": "2026-09-29T21:46:30+0000"
}
```

SQ-03 · Crítica · BACKEND/src/routes/pacientes.js:276

Regla javascript:S3776 — Se detecta complejidad cognitiva excesiva de la función (umbral permitido: 15). Las funciones que acumulan demasiadas decisiones encadenadas son difíciles de mantener y de probar. Recomendación: Extraer la lógica a funciones auxiliares de un solo propósito; además facilita las pruebas unitarias.

Fragmento de código real (BACKEND/src/routes/pacientes.js, líneas 273–282):

```
    273  });
    274
    275  // Actualizar paciente (médico, admin y paciente - solo sus propios datos)
>>  276  router.put("/:id", validate(updatePacienteSchema), async (req, res) => {
    277    try {
    278      const authUser = getUserFromAuthHeader(req);
    279      if (!authUser) {
    280        return res.status(401).json({
    281          error: "No autorizado",
    282          glosa: "Token inválido",
```

Registro JSON de la exportación oficial (extracto de Grupo1_SonarQube_issues.json):


```
{
  "key": "AaFGL_Pwh7pI0-4QFeO4",
  "rule": "javascript:S3776",
  "severity": "CRITICAL",
  "component": "cuidarte-plus:BACKEND/src/routes/pacientes.js",
  "project": "cuidarte-plus",
  "line": 276,
  "message": "Refactor this function to reduce its Cognitive Complexity from 21 to the 15 allowed.",
  "effort": "11min",
  "debt": "11min",
  "type": "CODE_SMELL",
  "creationDate": "2026-09-29T21:46:30+0000"
}
```


SQ-04 · Crítica · BACKEND/src/routes/users.js:283

Regla javascript:S3776 — Se detecta complejidad cognitiva excesiva de la función (umbral permitido: 15). Las funciones que acumulan demasiadas decisiones encadenadas son difíciles de mantener y de probar. Recomendación: Extraer la lógica a funciones auxiliares de un solo propósito; además facilita las pruebas unitarias.

Fragmento de código real (BACKEND/src/routes/users.js, líneas 280–289):

```
    280    }
    281  });
    282
>>  283  async function updateUsuarioEmpresa(
    284    req,
    285    res,
    286    authUser,
    287    { nombre_usuario, contrasena_hash, correo, rol_id }
```

```
  288  ) {
  289    try {
```

Registro JSON de la exportación oficial (extracto de Grupo1_SonarQube_issues.json):


```
{
  "key": "AaFGL_LNh7pI0-4QFeO1",
  "rule": "javascript:S3776",
  "severity": "CRITICAL",
  "component": "cuidarte-plus:BACKEND/src/routes/users.js",
  "project": "cuidarte-plus",
  "line": 283,
  "message": "Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed.",
  "effort": "17min",
  "debt": "17min",
  "type": "CODE_SMELL",
  "creationDate": "2026-09-29T21:46:30+0000"
}
```


SQ-05 · Crítica · BACKEND/src/routes/users.js:437

Regla javascript:S3776 — Se detecta complejidad cognitiva excesiva de la función (umbral permitido: 15). Las funciones que acumulan demasiadas decisiones encadenadas son difíciles de mantener y de probar. Recomendación: Extraer la lógica a funciones auxiliares de un solo propósito; además facilita las pruebas unitarias.

Fragmento de código real (BACKEND/src/routes/users.js, líneas 434–443):

```
    434    }
    435  }
    436
>>  437  async function updateUsuarioSistemaMedico(
    438    req,
    439    res,
    440    authUser,
    441    { nombre_usuario, contrasena_hash, correo, rol_id }
    442  ) {
    443    try {
```

Registro JSON de la exportación oficial (extracto de Grupo1_SonarQube_issues.json):


```
{
  "key": "AaFGL_LNh7pI0-4QFeO2",
  "rule": "javascript:S3776",
  "severity": "CRITICAL",
  "component": "cuidarte-plus:BACKEND/src/routes/users.js",
  "project": "cuidarte-plus",
  "line": 437,
```

```
  "message": "Refactor this function to reduce its Cognitive Complexity from 24 to the 15 allowed.",
  "effort": "14min",
  "debt": "14min",
  "type": "CODE_SMELL",
  "creationDate": "2026-09-29T21:46:30+0000"
}
```


SQ-06 · Crítica · FRONTEND/src/pages/examenes/ExamenForm.jsx:47

Regla javascript:S3776 — Se detecta complejidad cognitiva excesiva de la función (umbral permitido: 15). Las funciones que acumulan demasiadas decisiones encadenadas son difíciles de mantener y de probar. Recomendación: Extraer la lógica a funciones auxiliares de un solo propósito; además facilita las pruebas unitarias.

Fragmento de código real (FRONTEND/src/pages/examenes/ExamenForm.jsx, líneas 44–53):

```
    44    notas: yup.string().max(2000).optional(),
    45  });
    46
>>  47  export default function ExamenForm({ onSuccess, examenId }) {
    48    const { role } = useAuth();
    49    const [loading, setLoading] = useState(false);
    50    const [tiposExamen, setTiposExamen] = useState([]);
    51    const [pacientes, setPacientes] = useState([]);
    52    const [loadingData, setLoadingData] = useState(true);
    53    const [documentos, setDocumentos] = useState([]);
```

Registro JSON de la exportación oficial (extracto de Grupo1_SonarQube_issues.json):


```
{
  "key": "AaFGL_Qyh7pI0-4QFePK",
  "rule": "javascript:S3776",
  "severity": "CRITICAL",
  "component": "cuidarte-plus:FRONTEND/src/pages/examenes/ExamenForm.jsx",
  "project": "cuidarte-plus",
  "line": 47,
  "message": "Refactor this function to reduce its Cognitive Complexity from 33 to the 15 allowed.",
  "effort": "23min",
  "debt": "23min",
  "type": "CODE_SMELL",
  "creationDate": "2026-09-29T21:46:30+0000"
}
```


SQ-07 · Crítica · FRONTEND/src/pages/pacientes/PacienteForm.jsx:21

Regla javascript:S3776 — Se detecta complejidad cognitiva excesiva de la función (umbral permitido: 15). Las funciones que acumulan demasiadas decisiones encadenadas son difíciles

de mantener y de probar. Recomendación: Extraer la lógica a funciones auxiliares de un solo propósito; además facilita las pruebas unitarias.

Fragmento de código real (FRONTEND/src/pages/pacientes/PacienteForm.jsx, líneas 18–27):

```
      18      }
      19    }, [id]);
      20
>>    21    const loadPaciente = async () => {
      22      try {
      23        const { data } = await api.get(`/pacientes/${id}`);
      24        setInitialValues({
      25          rut: data.rut || "",
      26          primer_nombre: data.primer_nombre || "",
      27          apellido_paterno: data.apellido_paterno || "",
```

Registro JSON de la exportación oficial (extracto de Grupo1_SonarQube_issues.json):


```
{
  "key": "AaFGL_Rlh7pI0-4QFePj",
  "rule": "javascript:S3776",
  "severity": "CRITICAL",
  "component": "cuidarte-plus:FRONTEND/src/pages/pacientes/PacienteForm.jsx",
  "project": "cuidarte-plus",
  "line": 21,
  "message": "Refactor this function to reduce its Cognitive Complexity from 22 to the 15 allowed.",
  "effort": "12min",
  "debt": "12min",
  "type": "CODE_SMELL",
  "creationDate": "2026-09-29T21:46:30+0000"
}
```


SQ-08 · Crítica · FRONTEND/src/pages/usuarios/UsuarioForm.jsx:173

Regla javascript:S3776 — Se detecta complejidad cognitiva excesiva de la función (umbral permitido: 15). Las funciones que acumulan demasiadas decisiones encadenadas son difíciles de mantener y de probar. Recomendación: Extraer la lógica a funciones auxiliares de un solo propósito; además facilita las pruebas unitarias.

Fragmento de código real (FRONTEND/src/pages/usuarios/UsuarioForm.jsx, líneas 170–179):

```
     170      },
     171    });
     172
>>   173    useEffect(() => {
     174      if (!id) return;
     175      (async () => {
```

```
    176       try {
    177         // Cargar roles disponibles
    178         const rolesResponse = await api.get("/roles");
    179         setRoles(rolesResponse.data || []);
```

Registro JSON de la exportación oficial (extracto de Grupo1_SonarQube_issues.json):


```
{
  "key": "AaFGL_R3h7pI0-4QFePs",
  "rule": "javascript:S3776",
  "severity": "CRITICAL",
  "component": "cuidarte-plus:FRONTEND/src/pages/usuarios/UsuarioForm.jsx",
  "project": "cuidarte-plus",
  "line": 173,
  "message": "Refactor this function to reduce its Cognitive Complexity from 39 to the 15 allowed.",
  "effort": "29min",
  "debt": "29min",
  "type": "CODE_SMELL",
  "creationDate": "2026-09-29T21:46:30+0000"
}
```


SQ-09 · Mayor · BACKEND/src/routes/documentos.js:244

Regla javascript:S4165 — Se detecta asignación redundante: la variable ya contiene el valor que se le vuelve a asignar, por lo que la segunda asignación no tiene efecto. Recomendación: Eliminar la asignación sobrante y conservar solo la primera.

Fragmento de código real (BACKEND/src/routes/documentos.js, líneas 241–250):

```
    241     } else if (extension === 'png') {
    242       contentType = "image/png";
    243     } else if (extension === 'pdf') {
>>  244       contentType = "application/pdf";
    245     }
    246
    247     res.setHeader("Content-Type", contentType);
    248     res.setHeader(
    249       "Content-Disposition",
    250       `attachment; filename="${documento.nombre_archivo}"`
```

SQ-10 · Mayor · BACKEND/src/routes/users.js:60

Regla javascript:S4624 — Se detecta template literal anidado dentro de otro: complica la lectura y el mantenimiento del formato de texto. Recomendación: Construir el texto por partes o con una sola interpolación preformateada.

Fragmento de código real (BACKEND/src/routes/users.js, líneas 57–66):

```
57        "INSERT INTO auditoria (usuario_id, accion) VALUES ($1, $2)",
58        [
59          authUser?.usuarioId || null,
>>  60          `USUARIOS_CREAR: creó usuario ${nombre_usuario}${rol_id ? ` con rol ${rol_id}` :
""}`,
61        ]
62      );
63
64      res.status(201).json(newUser);
65    } catch (err) {
66      console.error(err);
```

SQ-11 · Mayor · FRONTEND/src/pages/pacientes/PacientesTabList.jsx:53

Regla javascript:S6478 — Se detecta componente React definido durante el render (dentro de otro componente): React lo trata como un tipo nuevo en cada render y desmonta/remonta el subárbol completo, con pérdida de estado y de rendimiento. Recomendación: Mover la definición del componente fuera del componente padre, o definirlo como constante fuera del render.

Fragmento de código real (FRONTEND/src/pages/pacientes/PacientesTabList.jsx, líneas 50–59):

```
50          accessorKey: "primer_nombre",
51          header: "Nombre Completo",
52          size: 250,
>>  53          Cell: ({ row }) => (
54            <Stack direction="row" spacing={1.5} alignItems="center">
55              <Avatar
56                sx={{
57                  bgcolor: "primary.main",
58                  width: 32,
59                  height: 32,
```

SQ-12 · Mayor · FRONTEND/src/pages/pacientes/PacientesTabList.jsx:86

Regla javascript:S6478 — Se detecta componente React definido durante el render (dentro de otro componente): React lo trata como un tipo nuevo en cada render y desmonta/remonta el subárbol completo, con pérdida de estado y de rendimiento. Recomendación: Mover la definición del componente fuera del componente padre, o definirlo como constante fuera del render.

Fragmento de código real (FRONTEND/src/pages/pacientes/PacientesTabList.jsx, líneas 83–92):

```
83          accessorKey: "email",
84          header: "Email",
```

```
      85          size: 200,
>>    86          Cell: ({ row }) => (
      87            <Stack direction="row" spacing={1} alignItems="center">
      88              <EmailIcon sx={{ fontSize: 16, color: "text.secondary" }} />
      89              <Typography variant="body2">{row.original.email || "-"}</Typography>
      90            </Stack>
      91          ),
      92        },
```

SQ-13 · Mayor · FRONTEND/src/pages/pacientes/PacientesTabList.jsx:97

Regla javascript:S6478 — Se detecta componente React definido durante el render (dentro de otro componente): React lo trata como un tipo nuevo en cada render y desmonta/remonta el subárbol completo, con pérdida de estado y de rendimiento. Recomendación: Mover la definición del componente fuera del componente padre, o definirlo como constante fuera del render.

Fragmento de código real (FRONTEND/src/pages/pacientes/PacientesTabList.jsx, líneas 94–103):

```
      94          accessorKey: "telefono",
      95          header: "Teléfono",
      96          size: 150,
>>    97          Cell: ({ row }) => (
      98            <Stack direction="row" spacing={1} alignItems="center">
      99              <PhoneIcon sx={{ fontSize: 16, color: "text.secondary" }} />
     100              <Typography variant="body2">
     101                {row.original.telefono || "-"}
     102              </Typography>
     103            </Stack>
```

SQ-14 · Mayor · FRONTEND/src/pages/pacientes/PacientesTabList.jsx:122

Regla javascript:S6478 — Se detecta componente React definido durante el render (dentro de otro componente): React lo trata como un tipo nuevo en cada render y desmonta/remonta el subárbol completo, con pérdida de estado y de rendimiento. Recomendación: Mover la definición del componente fuera del componente padre, o definirlo como constante fuera del render.

Fragmento de código real (FRONTEND/src/pages/pacientes/PacientesTabList.jsx, líneas 119–128):

```
     119          enableColumnFilter: false,
     120          enableSorting: false,
     121          enableColumnActions: false,
>>   122          Cell: ({ row }) => (
     123            <Stack direction="row" spacing={1} justifyContent="flex-end">
```

```
124              <IconButton
125                color="primary"
126                size="small"
127                data-testid={`button-cliente-editar-${row.original.id}`}
128                onClick={() => navigate(`/pacientes/${row.original.id}/editar`)}
```

SQ-15 · Mayor · FRONTEND/src/pages/auditoria/AuditoriaList.jsx:51

Regla javascript:S6478 — Se detecta componente React definido durante el render (dentro de otro componente): React lo trata como un tipo nuevo en cada render y desmonta/remonta el subárbol completo, con pérdida de estado y de rendimiento. Recomendación: Mover la definición del componente fuera del componente padre, o definirlo como constante fuera del render.

Fragmento de código real (FRONTEND/src/pages/auditoria/AuditoriaList.jsx, líneas 48–57):

```
48          accessorKey: "nombre_usuario",
49          header: "Usuario",
50          size: 200,
>>  51          Cell: ({ row }) => (
52            <Typography variant="body2">
53              {row.original.nombre_usuario || `ID: ${row.original.usuario_id || "N/A"}`}
54            </Typography>
55          ),
56        },
57        {
```

SQ-16 · Mayor · FRONTEND/src/pages/examenes/ExamenesTabList.jsx:71

Regla javascript:S6478 — Se detecta componente React definido durante el render (dentro de otro componente): React lo trata como un tipo nuevo en cada render y desmonta/remonta el subárbol completo, con pérdida de estado y de rendimiento. Recomendación: Mover la definición del componente fuera del componente padre, o definirlo como constante fuera del render.

Fragmento de código real (FRONTEND/src/pages/examenes/ExamenesTabList.jsx, líneas 68–77):

```
68                accessorKey: "paciente_primer_nombre",
69                header: "Paciente",
70                size: 250,
>>  71                Cell: ({ row }) => (
72                  <Typography variant="body2">
73                    {`${row.original.paciente_primer_nombre || ""} ${
74                      row.original.paciente_apellido_paterno || ""
75                    } ${row.original.paciente_apellido_materno || ""}`.trim()}
```

```
76                 </Typography>
77               ),
```

SQ-17 · Mayor · FRONTEND/src/pages/usuarios/UsuariosTabList.jsx:53

Regla javascript:S6478 — Se detecta componente React definido durante el render (dentro de otro componente): React lo trata como un tipo nuevo en cada render y desmonta/remonta el subárbol completo, con pérdida de estado y de rendimiento. Recomendación: Mover la definición del componente fuera del componente padre, o definirlo como constante fuera del render.

Fragmento de código real (FRONTEND/src/pages/usuarios/UsuariosTabList.jsx, líneas 50–59):

```
50          accessorKey: "nombre_usuario",
51          header: "Usuario",
52          size: 200,
>>  53          Cell: ({ row }) => (
54            <Stack direction="row" spacing={1.5} alignItems="center">
55              <Avatar
56                sx={{
57                  bgcolor: "primary.main",
58                  width: 32,
59                  height: 32,
```

SQ-18 · Mayor · FRONTEND/src/pages/examenes/ExamenForm.jsx:451

Regla javascript:S3358 — Se detecta operador ternario anidado (condicional dentro de un condicional), difícil de leer y de mantener. Recomendación: Reemplazar el ternario anidado por sentencias if/else o por una función auxiliar.

Fragmento de código real (FRONTEND/src/pages/examenes/ExamenForm.jsx, líneas 448–457):

```
448                         <Typography variant="h6" sx={{ mb: 1 }}>
449                           {isDragActive
450                             ? "Suelta el archivo aquí"
>>  451                             : uploading
452                             ? "Subiendo documento..."
453                             : "Arrastra un archivo aquí"}
454                         </Typography>
455                         <Typography variant="body2" color="text.secondary">
456                           o haz clic para seleccionar un archivo
457                         </Typography>
```

SQ-19 · Mayor · FRONTEND/src/pages/pacientes/PacientesTabCreate.jsx:501

Regla javascript:S3358 — Se detecta operador ternario anidado (condicional dentro de un condicional), difícil de leer y de mantener. Recomendación: Reemplazar el ternario anidado por sentencias if/else o por una función auxiliar.

Fragmento de código real (FRONTEND/src/pages/pacientes/PacientesTabCreate.jsx, líneas 498–507):

```
    498                   disabled={loading}
    499                 >
    500                   {loading
>>  501                     ? pacienteId
    502                       ? "Actualizando..."
    503                       : "Registrando..."
    504                     : pacienteId
    505                     ? "Actualizar Paciente"
    506                     : "Registrar Paciente"}
    507                 </Button>
```

SQ-20 · Mayor · FRONTEND/src/pages/usuarios/UsuariosTabCreate.jsx:188

Regla javascript:S3358 — Se detecta operador ternario anidado (condicional dentro de un condicional), difícil de leer y de mantener. Recomendación: Reemplazar el ternario anidado por sentencias if/else o por una función auxiliar.

Fragmento de código real (FRONTEND/src/pages/usuarios/UsuariosTabCreate.jsx, líneas 185–194):

```
    185                   <MenuItem key={rol.id} value={rol.id}>
    186                     {rol.nombre === "admin" || rol.nombre === "administrador"
    187                       ? "Administrador"
>>  188                       : rol.nombre === "medico"
    189                       ? "Médico"
    190                       : rol.nombre === "paciente"
    191                       ? "Paciente"
    192                       : rol.nombre}
    193                   </MenuItem>
    194                 ))}
```

SQ-21 · Mayor · FRONTEND/src/pages/informacion/Ayuda.jsx:119

Regla javascript:S6479 — Se detecta uso del índice del array como key en React: al reordenar o eliminar elementos de la lista, React puede montar el estado mal. Recomendación: Usar como key un identificador estable del dato (id, RUT) en lugar del índice.

Fragmento de código real (FRONTEND/src/pages/informacion/Ayuda.jsx, líneas 116–125):

```
116                </Typography>
117                <Stack spacing={2}>
118                  {faqGeneral.map((faq, index) => (
>> 119                    <Accordion key={index}>
120                      <AccordionSummary
121                        expandIcon={<ExpandMoreIcon />}
122                        aria-controls={`panel${index}-content`}
123                        id={`panel${index}-header`}
124                      >
125                        <Typography sx={{ fontWeight: 500 }}>
```

SQ-22 · Mayor · FRONTEND/src/components/RoleGuard.jsx:28

Regla javascript:S1854 — Se detecta asignación a una variable que nunca se vuelve a leer (código muerto). Recomendación: Eliminar la asignación o usar efectivamente la variable.

Fragmento de código real (FRONTEND/src/components/RoleGuard.jsx, líneas 25–34):

```
25  export function RequireAdminOrOwnUser({ children }) {
26    const { role, user } = useAuth()
27    const { id } = useParams()
>>  28    const location = useLocation()
29
30    // Si es administrador, permitir acceso
31    if (role === 'admin') return children
32
33    // Si es otro rol, solo permitir si está editando su propio usuario
34    if (user?.usuarioId && id && Number(user.usuarioId) === Number(id)) {
```

SQ-23 · Mayor · FRONTEND/src/pages/pacientes/PacienteForm.jsx:12

Regla javascript:S1854 — Se detecta asignación a una variable que nunca se vuelve a leer (código muerto). Recomendación: Eliminar la asignación o usar efectivamente la variable.

Fragmento de código real (FRONTEND/src/pages/pacientes/PacienteForm.jsx, líneas 9–18):

```
 9  export default function PacienteForm() {
10    const { id } = useParams();
11    const navigate = useNavigate();
>> 12    const [loading, setLoading] = useState(false);
13    const [initialValues, setInitialValues] = useState(null);
14
15    useEffect(() => {
16      if (id) {
17        loadPaciente();
18      }
```

SQ-24 · Mayor · FRONTEND/src/pages/pacientes/PacientesTabList.jsx:24

Regla javascript:S1854 — Se detecta asignación a una variable que nunca se vuelve a leer (código muerto). Recomendación: Eliminar la asignación o usar efectivamente la variable.

Fragmento de código real (FRONTEND/src/pages/pacientes/PacientesTabList.jsx, líneas 21–30):

```
21  import { useNavigate } from "react-router-dom";
22
23  export default function PacientesTabList({ items, onDelete }) {
>>  24    const { role } = useAuth();
25    const navigate = useNavigate();
26    const [openDeleteDialog, setOpenDeleteDialog] = useState(false);
27    const [pacienteToDelete, setPacienteToDelete] = useState(null);
28
29    const handleDeleteClick = useCallback((id, nombrePaciente) => {
30      setPacienteToDelete({ id, nombrePaciente });
```

SQ-25 · Mayor · FRONTEND/src/index.css:6

Regla css:S4649 — Se detecta la pila de fuentes no declara una tipografía genérica (p. ej. sans-serif): el texto puede renderizarse de forma inconsistente entre sistemas. Recomendación: Agregar una tipografía genérica al final de la pila de fuentes (p. ej. sans-serif).

Fragmento de código real (FRONTEND/src/index.css, líneas 3–12):

```
 3  @tailwind utilities;
 4
 5  :root {
>>  6    font-family: "Poppins", "Arial", "sans-serif";
 7    font-size: 14px; /* Reducido de 14px (default) a 12px (2pt menos) */
 8    line-height: 1.5;
 9    font-weight: 400;
10
11    font-synthesis: none;
12    text-rendering: optimizeLegibility;
```